

Priority Queue (PQ) intenal

(1)

The PQ is a data structure in which the intrinsic ordering of elements does determine the results of its basic operation.

There are two basic types of PQs: an ascending PQ and Descending PQ.

An ascending PQ is a collection of items in which items can be inserted arbitrarily and from which only the smallest item can be removed.

An descending PQ is similar but allows deletion of only largest item.

→ The elements of a priority queue need not be numbers or characters that can be compared directly. They may be complex structures that are ordered on one or several fields. For example, telephone book listings consist of last names, first names, addresses and phone numbers and are ordered by last name.

→ Sometimes, the field on which elements of a PQ is ordered is not even part of the elements themselves; it may be a special external value used specifically for the purpose of ordering the PQ.

For example, a stack may be viewed as descending PQ whose elements are ordered by time of insertion. The element that was inserted last has the greatest insertion time and is the only item that can be retrieved.

A Queue may similarly be viewed as an ascending Priority Queue whose elements are ordered by time of insertion.

In both the cases, time of insertion is not part of elements themselves, but it is used to order the PQ.

Array Implementation of PQ

As we have seen, a stack and queue can be implemented in an array such that each insertion or deletion involves accessing only a single element of the array. Unfortunately, this is not possible for a PQ.

Suppose that n -elements of a PQ (pq) are maintained in positions 0 to $n-1$ of an array pq of size maxpq. Suppose pqrear equals the first empty array position, n .

$pq \text{ insert}(pq, x)$ would seem to be fairly straight forward operation
 & if($pq \rightarrow \text{rear} \geq \text{max}(pq)$)
 {
 $\text{printf}("overflow/n");$
 $\text{exit}(1);$
 }
 {
 $pq \rightarrow \text{item}(pq \rightarrow \text{rear}) = x;$
 $pq \rightarrow \text{rear}++;$
 }

elements of
 PQ are not
 kept ordered in
 an array

As long as insertion takes place, this implementation works well.
 Suppose we attempt the operation $pq \text{ mindelete}(pq)$ on a APQ.
 This raises two issues: (1) firstly to locate the smallest element
 [every element of an array must be examined]. Therefore, a
 deletion requires to access every element of PQ.

(2) Secondly, how can an element in the middle of an array
 be deleted. Stack and Queue involve removal of an item from
 one of the two ends and don't searching. PQ deletion
 under above implementation requires both searching for
 element to be deleted and removal of element in the middle.

There are several solutions to this problem, none of them entirely
 satisfactory:

- (1) A special empty indicator can be placed into a deleted position. This
 indicator can be value that is invalid as a element or a separate
 field can be contained in each array element to indicate whether it is
 empty or not. There are several disadvantages of this approach.
 first, the search process to locate the min or max element must
 examine all deleted array positions in addition to actual PQ element.
 if many items have been deleted but not compaction has yet
 placed, the deletion operation may move array elements that
 exist in the PQ.

(2) The deleted operation labels a position empty, but insertion is modified to insert a new element in the first empty position. Insertion then involves accessing every array element up to first one that has been deleted. This ↓ the effcn of insertion is a major drawback to solⁿ.

(3) Each deletion can compact the array by shifting all elements by one position and then decrementing $pq \rightarrow rear$ by 1. Insertion remains unchanged. On average, the half of all PQ elements are shifted for each deletion so that deletion becomes quite inefficient. A slight better alternative is to shift all preceding elements forward or all succeeding elements backward, depending on which group is smaller.

(4) Instead of maintaining the PQ as an unordered array, maintain it as an ordered circular array. This will reduce the effort/expense on finding max or min of unordered array and a B. might be reduce the cost even more. [Heap sort]